Soft Stack Error Bounds

Bence Kaszás

May 19, 2026

1 Introduction

Here we want to formalise the proof on the memory fidelity in the binary string reversal task. The main goal is computing a bound on the expected symbol we output, and show that it decays towards random guessing with the length of the string.

2 Project scope and objectives

- binary string reversal
- abstract binary controller which at each time t has an error rate ϵ_t - meaning it puts probability mass $1 - \epsilon_t$ on the correct action (either push or pop).
- Potential different cases we could study for ϵ_t: consider ϵ_t fixed for all t, bounded for all t, distributed mutually iid w.r.t to some distribution.

3 Sketch of the proof

- try and sketch out the ballot counting based argument for the fixed $\epsilon_t = \epsilon$ case, think about how we should structure the argument, are there any issues we can foresee, are there anyways to generalize this to other problems
- How does the model we have proposed actually relate to a soft stack model which stores expected predictions?

We start with the fixed error rate case, where ϵ is constant across timesteps. Our goal is to compute a bound on the expected mass of the correct symbol.

3.1 Setup

First we define the stack indicators and variables. Let $\lambda(t, u)$ denote the position on the stack of the u-th object seen in the input buffer. Then we define the top of the stack as

$$B_u(t) \coloneqq \mathbf{1}(\text{element seen at time } u \text{ in the input sequence is at the top of the stack}).$$

This is equivalent to

$$B_u(t) \coloneqq \mathbf{1}(\lambda(t, u) = 1).$$

here the events $\{B_u(t)\}_{u \in \mathbb{N}}$ are mutually disjoint.
For the stack, let $m_k(t)$ represent the element at depth k at time t, and let $\xi(u) \in \Gamma$ be the input symbol at time u. We want to find the probability that the top element of the stack at time t, given by $m_1(t)$, is a specific target symbol $\gamma \in \Gamma$.

Here we also define another indicator, which represents the event that an element that was at time τ at depth k is at the top of the stack at time t. This is given by

$$\omega(t, k, \tau) \coloneqq \mathbf{1}(m_k(\tau) \text{ is at the top of the stack at time } t).$$

As we want to specifically use this indicator with τ fixed at the timestep of the phase switch, where we stop pushing to the stack and start popping of the elements, we can shorten this to $\omega(t, k)$, which implicitly assume a fixed $\tau = \frac{T}{2}$, where T is the total sequence length.
Using this, the total probability mass for the correct symbol γ being at the top of the stack can be decomposed into two sums, one representing probability mass coming from elements on the stack when we switch phases, and a second sum representing new elements being read in the pop phase. This is given by

$$\mathbb{P}(m_1(t) = \gamma) = \sum_{k=1}^{n+1} \mathbb{P}(\omega(t, k) = 1)\mathbb{P}(m_k(\tau) = \gamma) + \sum_{\tau \leq u \leq t} \mathbb{P}(\lambda(t, u) = 1)\mathbb{P}(\xi(u) = \gamma),$$

where n is the length of the input string.
Our goal in the proof is then to evaluate $\mathbb{P}(\omega(t, k) = 1)$, which can be represented as a ballot counting problem. In its most standard form, we want to calculate the number of integer valued random walks that consist of n steps of unit length, beginning at the origin and ending at m, while never becoming negative. We know that n and m have the same parity as well as that $n \geq m \geq 0$, and the number of paths is given by

$$N_{0 \to m,n} = \binom{n}{\frac{n+m}{2}} - \binom{n}{\frac{n+m}{2} + 1} = \frac{m+1}{n+m+1}\binom{n}{\frac{n+m}{2}}.$$

A special case of this is when $m = 0$, and the total number of possible paths is given by the Catalan number

$$\frac{1}{\frac{n}{2}+1}\binom{n}{\frac{n}{2}}.$$

3.2 Proof

Let N be the total lenght of the input sequence, and consider a target symbol pushed to the stack at time $t_i \in \{1, \dots n\}$ given by $\xi(t_i) \in \Gamma$. For a successful string reversal, this symbol needs to be at the top of the stack at time $t_{out} = 2N - t_i + 1$. Let $n = N - t_i$ be the number of timesteps in both the encoding and decoding phase for the given symbol.

Phase 1: encoding

As we assume a fixed ϵ throughout the proof, in the first phase we push onto the stack with probability $1 - \epsilon$ and pop off from it with probability ϵ. The number of valid paths from 0 to k in n steps is given by

$$N_{0 \to k,n} = \binom{n}{\frac{n+k}{2}} - \binom{n}{\frac{n+k}{2} + 1} = \frac{k+1}{n+k+1}\binom{n}{\frac{n+k}{2}}$$

Let U denote the number of pushes during the n timesteps, and D the number of pops. Then we have that $U + D = n$, as well as that $U - D = k$. This gives

$$2U = n + k \implies U = \frac{n+k}{2}, \text{ and } D = n - \frac{n+k}{2} = \frac{n-k}{2}.$$

Thus the probability that the symbol seen in the input at time t_i is at depth k at τ is given by

$$\mathbf{P}_{enc}(k) = \mathbb{P}(\lambda(\tau, t_i) = k) = \left[\binom{n}{\frac{n+k}{2}} - \binom{n}{\frac{n+k}{2} + 1}\right](1-\epsilon)^{\frac{n+k}{2}}\epsilon^{\frac{n-k}{2}}.$$

Phase 2: decoding

Similarly to the previous phase, here we assume a fixed ϵ, and push onto the stack with probability ϵ and pop off from it with probability $1-\epsilon$. Now, the number of valid paths from k to 0 in n steps is given by

$$N_{k\to 0,n} = \binom{n}{\frac{n-k}{2}} - \binom{n}{\frac{n-k}{2}-1}.$$

This is in fact equal to the number of paths in the encoding phase due to the symmetry of the binomial coefficient:

$$\binom{n}{\frac{n+k}{2}} = \binom{n}{n-(\frac{n+k}{2})} = \binom{n}{\frac{n-k}{2}}$$
$$\binom{n}{\frac{n+k}{2}+1} = \binom{n}{n-(\frac{n+k}{2}+1)} = \binom{n}{\frac{n-k}{2}-1}.$$

Similarly to the first phase, here the number of pop actions needed is $\frac{n+k}{2}$, and the number of push actions needed is $\frac{n-k}{2}$. Thus the probability of the element at depth k at time τ being at the top of the stack is given by

$$\mathbf{P}_{dec}(k) = \mathbb{P}(\omega(t,k)=1) = \left[\binom{n}{\frac{n+k}{2}} - \binom{n}{\frac{n+k}{2}+1}\right](1-\epsilon)^{\frac{n+k}{2}}\epsilon^{\frac{n-k}{2}}.$$

Total combined probability

As the two phases are independent random walks separated at time τ, the total probability that the correct symbol is at the top of the stack at time t_{out} is given by a sum over all the possible positions in the stack at time τ:

$$\begin{aligned}
\mathbb{P}(\lambda(t_{out},t_i)=1) &= \sum_{k=n \pmod 2} \mathbf{P}_{enc}(k)\cdot\mathbf{P}_{dec}(k) \\
&= \sum_{k=n \pmod 2} \left[\binom{n}{\frac{n+k}{2}} - \binom{n}{\frac{n+k}{2}+1}\right]^2 (1-\epsilon)^{n+k}\epsilon^{n-k} \\
&= \sum_{k=n \pmod 2} \left[\frac{k+1}{n+k+1}\binom{n}{\frac{n+k}{2}}\right]^2 (1-\epsilon)^{n+k}\epsilon^{n-k}.
\end{aligned}$$

We can bound this probability using Stirling's formula for the factorial, however the proof is not complete. This probability only accounts for the element at the correct source time t_i being at the top of the stack at t_{out}. For the total probability that the symbol at the top of the stack we have to also consider elements with matching symbols on the stack that were pushed at incorrect times, as well as elements erroneously pushed onto the stack at the decoding phase.

3.2.1 Accounting for correct symbols from erroneous times

We return to the total probability mass for the correct symbol being at the top of the stack at time t, which was stated above and is given by

$$\mathbb{P}(m_1(t)=\gamma) = \sum_{k=1}^{n+1}\mathbb{P}(\omega(t,k)=1)\mathbb{P}(m_k(\tau)=\gamma) + \sum_{\tau\le u\le t}\mathbb{P}(\lambda(t,u)=1)\mathbb{P}(\xi(u)=\gamma).$$

To decompose the first term, we can use the encoding and decoding probabilities $\mathbf{P}_{enc}(k)$ and $\mathbf{P}_{dec}(k)$ from above, by extending their definition to allow for not just a given stack depth k, but also the the number of

steps from the delimiter time τ. For the proof above, this distance was $n = \tau - t_i$ and $n = t_{out} - \tau$. We can denote this distance as d, and then we have that the probability of an element being at depth k at time τ given that it was seen at time $\tau - d$ is given by

$$\mathbf{P}_{enc}(k,d) = \left[\binom{d}{\frac{d+k}{2}} - \binom{d}{\frac{d+k}{2}+1}\right](1-\epsilon)^{\frac{d+k}{2}}\epsilon^{\frac{d-k}{2}},$$

and

$$\mathbf{P}_{dec}(k,d) = \left[\binom{d}{\frac{d+k}{2}} - \binom{d}{\frac{d+k}{2}+1}\right](1-\epsilon)^{\frac{d+k}{2}}\epsilon^{\frac{d-k}{2}}.$$

We can then decompose the first term in the probability above into a term representing the algorithmic fidelity, which is equal to $\mathbb{P}(\lambda(t_{out}, t_i) = 1)$, and a term accounting for the correct symbol coming from wron source times. The two terms are then given by

$$\begin{aligned}\mathbb{P}(m_1(t) = \gamma) = &\sum_{k=1}^{n+1} \mathbf{P}_{dec}(k,n)\mathbf{P}_{enc}(k,n) \\ &+ \sum_{k=1}^{n+1} \mathbf{P}_{dec}(k,n) \sum_{j \neq t_i} \mathbf{P}_{enc}(k, N-j)\mathbb{P}(\xi(j) = \gamma) \\ &+ \sum_{\tau \leq u \leq t} \mathbb{P}(\lambda(t,u) = 1)\mathbb{P}(\xi(u) = \gamma).\end{aligned}$$

$$\begin{aligned}\mathbb{P}(m_1(t) = \gamma) = &\sum_{k=n \pmod 2} \left[\frac{k+1}{n+k+1}\binom{n}{\frac{n+k}{2}}\right]^2 (1-\epsilon)^{n+k}\epsilon^{n-k} \\ &+ \frac{1}{2}\sum_{k=1}^{n+1}\left[\binom{n}{\frac{n+k}{2}} - \binom{n}{\frac{n+k}{2}+1}\right](1-\epsilon)^{\frac{n+k}{2}}\epsilon^{\frac{n-k}{2}} \sum_{j \neq t_i}\left[\binom{N-j}{\frac{N-j+k}{2}} - \binom{N-j}{\frac{N-j+k}{2}+1}\right](1-\epsilon)^{\frac{N-j+k}{2}}\epsilon^{\frac{N-j-k}{2}} \\ &+ \frac{1}{2}\sum_{\tau \leq u \leq t} \underbrace{\frac{1}{\frac{t-u}{2}+1}\binom{t-u}{\frac{t-u}{2}}}_{C_{\frac{t-u}{2}}}.\end{aligned}$$

Third term

For the third term, accounting for the

4 Open questions

- why binary string reversal?
- what statistics could one compute from any controller model to estimate in some sense its ϵ_t characteristics and hence predict its length gen failure due to memory fidelity. The nicest example would probably that uses the in horizon training or generalization bound.
- can we use online statics, i.e. during inference we could measure and track the ϵ rates and then feed these into a dynamic program to give us online some notion of confidence based on our predictions due to memory fidelity.
- what are the prod and cons of removing the issues of the soft stack say just by hardening the stack actions? is this a good fix?